

CMSC216: Practice Final Exam B

Spring 2026, University of Maryland

Exam period: 20 minutes *Points available:* 40

Problem 1 (20 pts): Nearby is a serial function called `array_average()`. Write a Threaded version of this function that does not use any global variables. For full credit, utilize a mutex to coordinate change to shared data.

```
1 float array_average_serial(float *arr, int len){
2     float sum = 0.0;
3     for(int i=0; i<len; i++){
4         sum += arr[i];
5     }
6     return sum / len;
7 }
```

Problem 2 (20 pts): Below are two functions that augment El Malloc with the block shrinking; this allows a user to specify that the originally requested size for a memory area can be adjusted down potentially creating open space. Fill in the definitions for these functions.

```
el_blockhead_t *el_shrink_block(el_blockhead_t *head, size_t newsize){
// Shrinks the size of the given block potentially creating a new block. Computes remaining space
// as the difference between the current size and parameter newsize. If this is smaller than
// EL_BLOCK_OVERHEAD, does nothing further and returns NULL. Otherwise, reduces the size of the
// given block by adjusting its header and footer and establishes a new block above it with
// remaining space beyond the block overhead. Returns a pointer to the newly introduced blocks. Does
// not modify any links in lists.
```

```
}
int el_shrink(void *ptr, size_t newsize){
// Shrink the area associated with the given ptr if possible. Checks to ensure that the block
// associated with the given user ptr is EL_USED and exits if not. Uses el_shrink_block() to
// adjust the block size and create a block for the remaining space. If not possible to shrink,
// returns 0. Otherwise moves the current block to the front of the Used List and places the newly
// created block to the front of the Available List after setting its state to EL_AVAILABLE. Returns
// 1 on successfully shrinking.
```

```
}
```